

Offline-First Hybrid Transaction Envelopes for Secure Daily Petty-Cash Transactions with Deferred Submission and Duress Authorization

Author name(s) withheld for review

Affiliation withheld for review

Abstract

This paper proposes a transaction-level hybrid cryptographic envelope for daily petty-cash and mobile e-payment systems, designed around *offline-first* operation. For every payment, the client creates a fresh ephemeral P-256 elliptic-curve Diffie-Hellman (ECDH) key pair and combines it with an authenticated long-term bank ECDH public key. HKDF-SHA256 derives a transaction-specific AES-256 key, while AES-256-GCM protects the confidential payment payload and selected associated metadata. A device-bound ECDSA P-256 signing key, released only after operating-system biometric authorization, signs the canonical protected envelope and provides sender-specific authorization. The same authorization step is extended, through key separation and at no added cryptographic cost, to a restricted *duress* authorization path. Timestamp, nonce, and a globally unique transaction identifier support freshness, while an atomic server-side check-and-commit rule prevents duplicate and concurrent settlement. Because daily petty-cash usage is inherently subject to intermittent connectivity, the same protected envelope is designed first for *offline creation*, *protected local queuing*, and *key-validity-aware deferred submission*; we detail the offline lifecycle, the interaction between queued envelopes and receiver-key revocation, and local-storage protection for pending transactions. The principal trade-off is that the receiver uses a long-term ECDH key to permit asynchronous envelope construction; therefore the design does not provide full forward secrecy if that private key is later compromised.

Index Terms—e-payment, hybrid cryptography, ECDH, HKDF, AES-256-GCM, ECDSA, biometric authorization, replay prevention, idempotency, offline-first payment, deferred submission, duress authorization.

1 Introduction

Banking and petty-cash transactions contain sensitive values such as sender, receiver, amount, time, and transaction identifiers. Encryption alone is insufficient: a payment service must also detect modification, authenticate the authorizing user, reject replayed or duplicated requests, protect device keys, *tolerate temporary connectivity loss*, and preserve correct account state under concurrent processing. Mobile-payment research increasingly combines transaction-time authorization, platform-backed key protection, and protocol-level replay controls [5, 6].

This work integrates those requirements into one signed

transaction envelope, with intermittent connectivity treated as the *normal* operating condition for daily petty-cash use rather than an exception. The design makes four main contributions: (1) an ephemeral-sender/static-receiver ECDH key-establishment path with explicit HKDF context binding; (2) AES-256-GCM authenticated encryption for the payment payload; (3) biometric-gated, device-bound ECDSA authorization over the exact protected envelope, extended via key separation to a restricted duress path with no additional cryptographic operation; and (4) an offline-first lifecycle for the same signed envelope — creation, encrypted local queuing, and key-validity/revocation-aware deferred submission — together with freshness and atomic idempotency for ordinary and concurrent replay resistance. The contribution is protocol and system integration rather than a new cryptographic primitive.

2 System and Threat Model

The protocol contains a client device, an enrolled user, and a receiver service. The receiver owns a long-term P-256 ECDH key pair (SK_R^{dh}, PK_R^{dh}) identified by *KeyID*. The public key is authenticated through the application trust path, while the private key is intended to reside in an HSM or equivalent isolated service. Each enrolled device owns a distinct P-256 ECDSA signing pair (SK_U^{sig}, PK_U^{sig}) ; §3.1 extends this to a second, policy-restricted pair. Signing private keys are non-exportable where supported and require per-operation user authentication through the platform keystore and biometric prompt [5].

The network adversary may eavesdrop, record, modify, drop, reorder, duplicate, or replay application messages. The model also considers sender impersonation, bank-key substitution attempts, duplicate/concurrent request delivery, and opportunistic, time-constrained coercion of the user. A fully compromised server/HSM, a fully malware-controlled device able to invoke authorized operations after authentication, denial-of-service, and coercion outcomes beyond application-level restrictions remain outside the cryptographic guarantees. TLS remains mandatory; the transaction envelope is an application-layer protection mechanism, not a replacement for transport security [7].

3 Cryptographic Transaction Architecture

ECDH keys are used only for key establishment and ECDSA keys only for signatures, following key-separation practice [1,

4]. A payment message is

$$M = \{S, R, A, T, N, TxID\}, \quad (1)$$

where S and R are sender and receiver identifiers, A is the amount, T is the timestamp, N is a fresh nonce, and $TxID$ is a globally unique transaction identifier.

For each transaction, the sender generates a fresh ephemeral P-256 ECDH pair (esk, ePK) and computes

$$Z = \text{ECDH}(esk, PK_R^{dh}). \quad (2)$$

The ECDH result is not used directly as an AES key. Instead,

$$\text{salt} = H(\text{HTE-v1-salt} \parallel S \parallel TxID \parallel N), \quad (3)$$

$$PRK = \text{HKDF-Extract}(\text{salt}, Z), \quad (4)$$

$$\text{info} = \text{HTE-v1/AES-256-GCM} \parallel T \parallel ePK \parallel KeyID, \quad (5)$$

$$K_T = \text{HKDF-Expand}(PRK, \text{info}, 32). \quad (6)$$

The 32-byte output K_T is the transaction-specific AES-256 key [2].

The sender generates a fresh 96-bit GCM IV and defines

$$AAD = \{v, S, T, N, TxID, KeyID, ePK\}, \quad (7)$$

where v is the protocol version. The confidential payment content is protected as

$$(C, Tag) = \text{AES-256-GCM}_{K_T}(IV, M, AAD), \quad (8)$$

so successful tag verification authenticates both the ciphertext-protected content and the AAD [3]. After the user confirms the displayed transaction details, biometric authorization releases one ECDSA signing operation:

$$Sig = \text{ECDSA-SHA256}_{SK_U^{sig,k}}(v \parallel KeyID \parallel ePK \parallel IV \parallel C \parallel Tag \parallel AAD), \quad (9)$$

with $k \in \{\text{normal}, \text{duress}\}$ selected by the platform authenticator, as detailed in §3.1. The transmitted or queued package is

$$P = \{v, KeyID, AAD, ePK, IV, C, Tag, Sig\}. \quad (10)$$

On receipt of P , the receiver performs the following ordered checks: (1) validate protocol version and $KeyID$; (2) verify the ECDSA signature against the registered device public-key set $\{PK_U^{normal}, PK_U^{duress}\}$ and record which key matched; (3) verify timestamp policy, nonce syntax/state, and $TxID$ format; (4) reject or idempotently return a previously committed $TxID$; (5) compute the corresponding ECDH shared secret and derive K_T using the identical HKDF encoding; (6) verify the GCM tag and decrypt M ; (7) validate amount, account state, limits, receiver, and authorization policy — applying the restricted duress profile of §3.1 if step (2) matched PK_U^{duress} ; and (8) atomically commit the financial update together with the unique $TxID$ record. No balance-changing side effect occurs before all cryptographic and business-rule checks succeed.

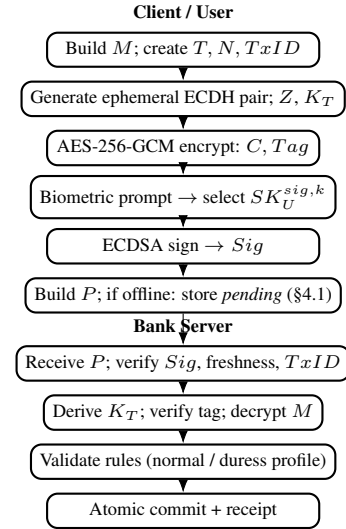


Figure 1: End-to-end transaction flow. The sender builds, encrypts, and signs the protected envelope (the signing key selects normal or duress profile); the receiver verifies, derives the same key, decrypts, validates, and atomically commits.

3.1 Duress Authorization via Key Separation

Routine payment authorization links two facts: the operating system verifies an allowed local authenticator, and successful authorization permits one ECDSA signing operation with the enrolled device key. To support restricted duress operation *without* adding a cryptographic operation or a network round trip, each device additionally enrolls a second, policy-restricted ECDSA key pair $(SK_U^{sig,duress}, PK_U^{sig,duress})$, gated by a distinct platform authenticator trigger (e.g., a separate enrolled fingerprint or a distinct device credential mapped to a separate keystore alias). Signing equation (9) is unchanged in form: only the enrolled key selected by the OS authenticator at signing time differs. Because key selection happens entirely inside the platform keystore/secure enclave, no extra ECDH, HKDF, or AES-GCM operation is introduced on the client, and the receiver’s added cost is a single extra public-key lookup against a two-entry per-device table at step (2) of §4 — an $O(1)$ addition with no measurable latency impact.

On a match against $PK_U^{sig,duress}$, the receiver applies a restricted wallet/profile with a preconfigured spend limit L_D ; transactions within this profile are real but bounded, and activation creates a silent risk event for later review and may temporarily restrict sensitive operations.

We state the scope of this mechanism explicitly, consistent with Clark and Hengartner’s analysis of panic-password schemes [8]: key separation gives a *policy-level, non-indistinguishable* duress signal. It does not defeat a patient adversary who is aware the mechanism exists and who can compel disclosure of both enrolled credentials, nor does it address lawful compelled-disclosure scenarios. The mechanism is scoped to opportunistic, time-constrained coercion (e.g., street-level robbery), where the adversary cannot verify which credential was used and cannot feasibly obtain both before the transaction must complete. This limitation is inherent

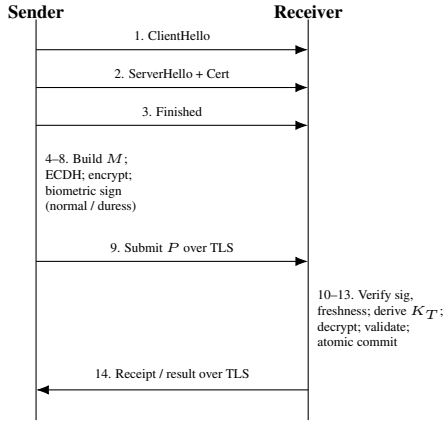


Figure 2: Compact online transaction sequence using the same protected envelope.

to duress-code systems in general and is not specific to this design.

4 Operational Capabilities

Figure 2 summarizes the ordinary online path: the sender builds and signs the protected package, submits it immediately, and the receiver performs freshness, signature, decryption, business-rule, and atomic settlement checks. OTP may remain available for account recovery, new-device enrollment, high-risk escalation, or policy exceptions; the receiver validates the resulting signature, not the biometric itself.

4.1 Offline-First Deferred Submission

Because daily petty-cash payments routinely occur in low- or intermittent-connectivity settings (markets, transit, rural branches), offline handling is treated as a primary design target rather than a fallback. Offline capability is provided through *offline creation and protected queuing with later submission*, not offline settlement; final settlement always remains receiver-authoritative.

Key-validity caching. A production deployment attaches *KeyID*, a validity interval $[T_{start}, T_{end}]$, and algorithm/version metadata to each receiver ECDH public key. A sender that already holds a valid, authenticated $(KeyID, PK_R^{dh})$ can construct the cryptographic envelope without a live ECDH round trip. To bound staleness, the cached key record itself carries a local cache lifetime $\tau_{cache} \leq (T_{end} - T_{start})$; once τ_{cache} elapses the client must refresh the record online before constructing further envelopes, even if T_{end} has not yet passed. This allows a bounded run of offline transactions while preventing unbounded use of a key the sender can no longer verify is current.

Local queue protection and ordering. Each queued package P is stored in platform secure storage (keystore-backed encryption at rest), not merely local storage, so that a lost or stolen device does not expose queued payment details prior to authentication. Each queued P additionally carries a device-local monotonic sequence number seq_i , distinct from $TxID$ and never used for server-side uniqueness, so the client can detect gaps or corruption in the local queue at resynchronization

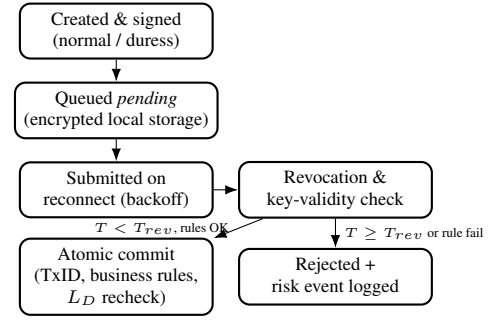


Figure 3: Offline transaction lifecycle. The signed envelope is created and queued locally during loss of connectivity; on resubmission the receiver checks key validity as of the envelope’s creation time before applying business rules and committing atomically.

time. Uniqueness and duplicate-settlement prevention remain based solely on the server’s atomic $TxID$ check.

Deferred submission and retry. When connectivity returns, the same immutable P and the same $TxID$ are submitted, using exponential backoff on failure. Retrying does not create a new transaction identity; the receiver’s unique atomic $TxID$ check therefore prevents repeated settlement without any client-side deduplication logic. The queued transaction may still be rejected later because of balance, risk, key-validity, or business-rule changes.

Revocation during pending state. Because an envelope may be signed well before it is submitted, the receiver resolves key validity against the envelope’s *creation* time T , not the submission time. Let T_{rev} be the receiver-logged revocation timestamp for the signing key that produced Sig . If $T < T_{rev}$, the envelope is evaluated under the applicable grace-period policy (below); if $T \geq T_{rev}$, the envelope is rejected and a risk event is logged, since a signature created after revocation cannot be trusted regardless of client-reported timestamps. During key rotation, a bounded grace period may retain retired receiver private keys only long enough to process already-queued envelopes referencing the retired *KeyID*; new envelopes always use the current key. Emergency revocation invalidates compromised identifiers and forces online re-synchronization before further envelope creation.

Offline duress bound. An offline-created duress envelope cannot query the live remaining balance of the restricted profile’s spend limit L_D . The client therefore maintains a conservative local counter against L_D , decremented per queued duress envelope, in addition to the receiver’s authoritative recheck at commit time; a mismatch (e.g., L_D already consumed through another channel) causes rejection at commit with a logged risk event, never a silent overspend.

Figure 3 summarizes this lifecycle.

The client ECDH key is ephemeral, but the receiver ECDH key is long-term. This supports asynchronous and offline-capable envelope construction because a validated receiver public key can be cached. The cost is that the scheme does not provide full forward secrecy against later compromise of SK_R^{dh} : an attacker that recorded an old ePK and envelope could, after obtaining the receiver private key, recompute the historical shared secret and derive the old transaction key.

The elliptic-curve envelope also has lower public-key overhead than an illustrative RSA-2048 key-transport plus RSA-PSS construction. With compressed P-256 public-key encoding (33 bytes), a 96-bit GCM IV (12 bytes), a 128-bit tag (16 bytes), and a raw P-256 ECDSA $r||s$ signature (64 bytes), fixed cryptographic overhead is approximately 125 bytes, excluding AAD, identifiers, framing, and serialization; this is an encoding-level estimate, not an implementation benchmark.

The design has explicit limits. Hardware-backed key protection reduces extraction risk but does not make a fully compromised endpoint trustworthy. Biometric authorization proves local user presence according to platform policy; the receiver's cryptographic evidence remains the ECDSA signature. The envelope does not prevent denial of service, traffic analysis, endpoint malware, receiver compromise, or fraudulent transactions that are cryptographically valid but socially engineered. Offline capability is deferred submission, not offline settlement. Secure deployment further depends on correct randomness, canonical encoding, TLS, certificate/key validation, HSM/Keystore configuration, database transactions, monitoring, and key-revocation procedures [7].

5 Implementation and Evaluation

[Placeholder — to be completed by the author with prototype details.] A reference prototype (mobile client and back-end receiver implementing §4–§4.1) is in progress. This section will report: end-to-end online transaction latency; the marginal cost, if any, of the duress key-selection branch relative to normal-path signing and verification; local secure-storage overhead per queued offline envelope; and resynchronization behavior (backoff schedule, commit/rejection rates) under simulated intermittent-connectivity traces representative of daily petty-cash use.

6 Conclusion

The proposed framework combines fresh per-transaction ECDH/HKDF key establishment, AES-256-GCM authenticated encryption, and biometric-gated device ECDSA authorization in one protected transaction envelope, extended to a restricted duress path through key separation at no added cryptographic cost. Timestamp, nonce, unique transaction identifiers, and atomic receiver idempotency address replay and duplicate settlement. Because daily petty-cash use is characteristically subject to intermittent connectivity, the same immutable package is designed offline-first: created, queued in encrypted local storage, and resubmitted with explicit, creation-time-aware handling of receiver-key revocation, without introducing independent ad-hoc cryptographic paths. The design is intentionally explicit about the static-receiver-key forward-secrecy trade-off, about the non-indistinguishable nature of the duress mechanism, and about the fact that final settlement remains receiver-authoritative.

References

- [1] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," NIST SP 800-56A Rev. 3, Apr. 2018.
- [2] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF RFC 5869, May 2010.
- [3] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, Nov. 2007.
- [4] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, Feb. 2023.
- [5] R. Mayrhofer, J. Vander Stoep, C. Brubaker, D. Hackborn, B. Bonné, G. S. Tuncay, R. Piqueras Jover, and M. A. Specter, "The Android Platform Security Model (2023)," arXiv:1904.05572, updated Dec. 2023.
- [6] W. Ahmed, A. Rasool, A. R. Javed, N. Kumar, T. R. Gadekallu, Z. Jalil, and N. Kryvinska, "Security in Next Generation Mobile Payment Systems: A Comprehensive Survey," *IEEE Access*, vol. 9, pp. 115932–115950, 2021.
- [7] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 9846, Jul. 2026.
- [8] J. Clark and U. Hengartner, "Panic Passwords: Authenticating under Duress," in *Proc. 3rd USENIX Workshop on Hot Topics in Security (HotSec '08)*, San Jose, CA, Jul. 2008.

Table 1: Threat-to-control mapping for the proposed framework.

Threat / Failure Mode	Primary Mechanism(s)	How the Framework Responds
Payload disclosure / passive eavesdropping	Per-transaction ECDH + HKDF-SHA256 + AES-256-GCM	Sender and receiver derive the same fresh K_T without transmitting it; an observer sees only public envelope metadata and ciphertext.
Ciphertext, IV, AAD, or metadata tampering	AES-GCM authentication tag + ECDSA signature	GCM rejects unauthorized modification of protected plaintext/AAD; ECDSA additionally binds ePK , KeyID, IV, ciphertext, tag, and freshness metadata.
Masquerade / sender impersonation	Registered device ECDSA public key + non-exportable private key + biometric-gated key use	A party without the enrolled private signing key cannot produce a valid authorization signature; the receiver verifies against the registered device public-key set.
Replay of an old valid transaction	Timestamp T + nonce N + unique $TxID$ + freshness policy	Stale requests are rejected by policy; previously committed identifiers are handled idempotently instead of being settled again.
Concurrent replay / duplicate settlement	Database uniqueness on $TxID$ + atomic check-and-commit	Simultaneously arriving valid duplicates cannot race into multiple balance updates.
Receiver-key substitution / active MITM against ECDH	Authenticated receiver ECDH public key + KeyID + TLS/certificate validation	The sender accepts only a receiver key delivered through the authenticated trust path, bound into HKDF context, AAD, and the signed envelope.
Stolen app files / private-key extraction attempt	Platform Keystore/StrongBox + non-exportable ECDSA key + per-operation authentication	Copying application storage should not reveal the private signing key when hardware-backed storage is available.
Connectivity loss, retry storms, or duplicate resubmission	Immutable signed pending envelope + same $TxID$ + receiver idempotency + backoff	The sender resends one signed identity; retries do not mint new identifiers, so repeated synchronization converges on one receiver result.
Key revocation during offline queue window	Creation-time validity check against T_{rev} + grace-period policy	Envelopes signed before revocation are handled per grace policy; envelopes signed at/after revocation are rejected and flagged.
Forced-access / coercive-use scenario	Restricted duress key/profile + pre-configured limit + silent risk event	The session exposes only restricted account state and bounded real transfers; scope limited to opportunistic, time-constrained coercion (§3.1).

Table 2: Comparison with related mobile-/card-payment security mechanisms.

Dimension	EMV (chip/contactless)	PCI Tokenization	Signal X3DH	This work (HTE)
Key establishment	Static symmetric/asymmetric card keys, issuer-derived	None — token vault replaces PAN	Ephemeral-ephemeral + one-time prekeys (X25519)	Ephemeral (sender) + static (receiver) ECDH, P-256
Payload confidentiality	Card/issuer MAC or encryption (scheme-dependent)	Token carries no payment data; PAN encrypted at vault	Double Ratchet (AES + HMAC chains)	AES-256-GCM with HKDF-derived per-transaction key
Sender authorization	PIN / signature / CDA cryptogram	Merchant/API credential, not sender-biometric	Identity-key signature at session start only	Biometric-gated ECDSA over the exact envelope, every transaction
Replay / duplicate protection	Application transaction counter	Token single-use or vault-side dedup	Message counters / ratchet state	Timestamp + nonce + unique $TxID$ + atomic check-and-commit
Offline capability	Offline data authentication, limited	N/A (online vault lookup)	Not designed for offline queuing	Explicit offline creation, encrypted local queuing, revocation-aware deferred submission
Duress / coercion handling	Not specified	Not specified	Not specified	Restricted-profile duress key with spend limit + silent risk event (scope-limited, §3.1)
Forward secrecy	N/A	N/A	Strong (per-message ratchet)	Partial — broken by static receiver ECDH key compromise
Formal verification	Long-standing, field-tested	Industry-audited (PCI DSS)	Formally analyzed in literature	Not yet performed (future work)